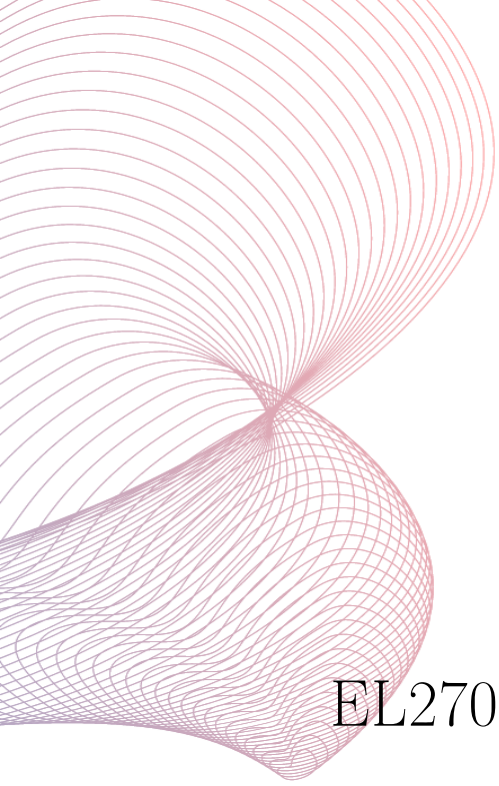
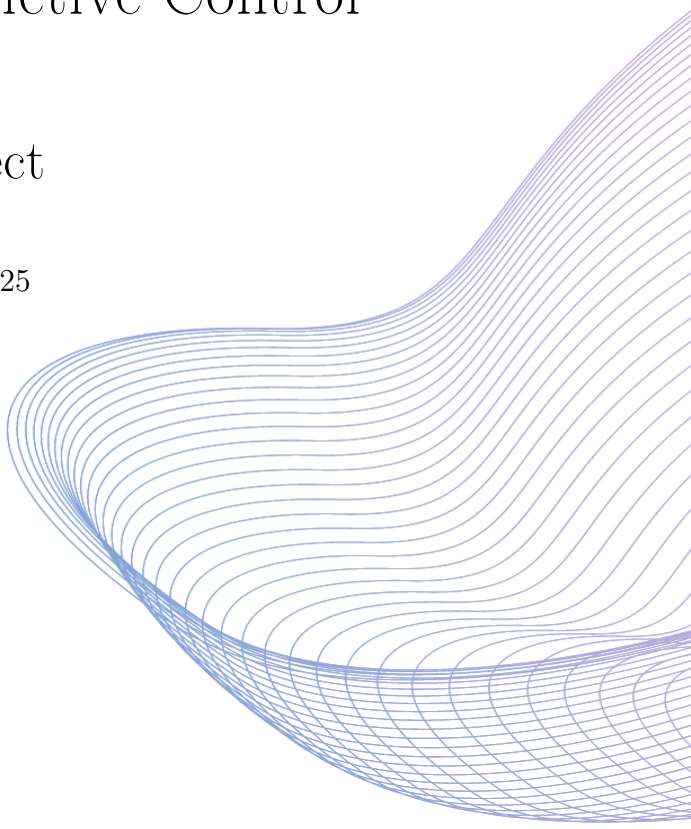


EL2700 - Model Predictive Control

Final Project

September 25, 2025



Final Project

Gregorio Marchesini, Samuel Erickson Andersson, and Mikael Johansson

Division of Decision and Control Systems
School of Electrical Engineering and Computer Science
Kungliga Tekniska Högskolan

Introduction

After this long road to the top, you finally made it into the F1 tour, and today you will be competing with your teammates for the glory. Your task for today will be to design the fastest raceline you can achieve with your given vehicle and track it using a fully Nonlinear Model Predictive Controller. In all the previous assignments, you have worked in the convex world—the dynamics of your system was linear, the constraints on your system were linear, and your objective function was convex. But now, when things get serious, it is time to use real nonlinear models.

The objective is simple: drive as fast as you can. But as you will see, even this simple goal is not at all easy when dealing with a nonlinear system. For this final project, we will use the kinematic bicycle model of the vehicle that we introduced in the first assignment:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\phi} \\ \dot{v}_x \\ \dot{v}_y \\ \dot{\omega} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} v_x \cos(\phi) - v_y \sin(\phi) \\ v_x \sin(\phi) + v_y \cos(\phi) \\ \omega \\ \frac{1}{m}(F(u_t) - \frac{1}{2} \cdot (v_x^2 + v_y^2) \cdot C_d \cdot A_{ref} \cdot \rho \\ (\dot{\delta} v_x + \dot{v}_x \delta) \cdot \frac{l_r}{l_r + l_f} \\ (\dot{\delta} v_x + \dot{v}_x \delta) \cdot \frac{1}{l_r + l_f} \\ u_\delta \end{bmatrix} \tag{1}$$

Here, the driving force is taken to be the following function of the throttle input $u_t \in [-1, 1]$

$$F(u_t) = F_{\min} + (F_{\max} - F_{\min}) \cdot (u_t + 1)/2 \tag{2}$$

where $F_{\min}$ is the maximum breaking force (obtained for $u_t = -1$) and $F_{\max}$ is the maximum acceleration force (obtained for $u_t = 1$). Moreover, we here have introduced the aerodynamic drag force

$$F_d = \frac{1}{2} \cdot (v_x^2 + v_y^2) \cdot C_d \cdot A_{\text{ref}} \cdot \rho.$$

Note that we have changed the yaw rate notation from r to ω due to some naming convention in the code for this assignment. So please, be careful with this notation change.

To design a racing line that is as fast as possible, the dynamics in (4) need to be augmented with three extra coordinates that represent the current state of the vehicle with respect to the

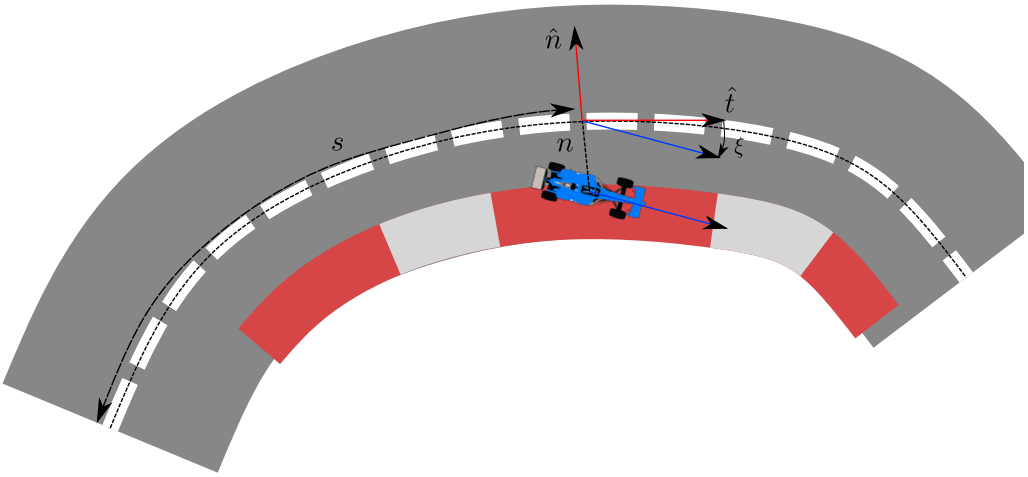


Figure 1: Local coordinates of the vehicle along the path

centerline. Namely, given the Frenet frame of the centerline, the progress of the vehicle with respect to the centerline is defined by the coordinates:

- The intrinsic coordinate of the vehicle along the centerline s,
- The lateral displacement n,
- The orientation of the vehicle with respect to the centerline tangent expressed by the angle ξ.

You can see this represented in Figure 1. The dynamics of the intrinsic coordinates is

$$\begin{aligned} \dot{s} &= \frac{v_x \cos(\xi) - v_y \sin(\xi)}{1 - n\kappa(s)} \\ \dot{n} &= v_x \sin(\xi) + v_x \cos(\xi) \\ \dot{\xi} &= \omega - \dot{s}\kappa(s) \end{aligned} \tag{3}$$

Minimum lap-time raceline optimization: Since the inertial position of the vehicle is not as important as the coordinate of the system in the Frenet frame, we consider the nonlinear system that includes the dynamics of the coordinates n and ξ (the coordinate s will be used as a cost as we will see shortly).

$$\dot{x} = \begin{bmatrix} \dot{\phi} \\ \dot{v}_x \\ \dot{v}_y \\ \dot{\omega} \\ \dot{\delta} \\ \dot{n} \\ \dot{\xi} \end{bmatrix} = \begin{bmatrix} \omega \\ \frac{1}{m}(F(u_t) - \frac{1}{2} \cdot (v_x^2 + v_y^2) \cdot C_d \cdot A_{ref} \cdot \rho \\ (\dot{\delta} v_x + \dot{v}_x \delta) \cdot \frac{l_r}{l_r + l_f} \\ (\dot{\delta} v_x + \dot{v}_x \delta) \cdot \frac{1}{l_r + l_f} \\ u_\delta \\ v_x \sin(\xi) + v_x \cos(\xi) \\ r - \dot{s}\kappa(s) \end{bmatrix} = f_t(x, u, \kappa) \tag{4}$$

Remembering that the coordinate s marks the progress of the vehicle along the racetrack, the lap time of the vehicle is given by the integral

$$T = \int_0^S \frac{1}{\dot{s}(s)} ds \tag{5}$$

where S is the total length of the racetrack. Moreover, we remember that the vehicle is subject to a set of state and input constraints $g_i^x(x,\kappa) \leq 0$ and $g_i^u(x,\kappa) \leq 0$ with

$$\begin{aligned}
g_1^x(x,\kappa) &= v_x^2 + v_y^2 - v_{\max}^2 && \text{(Maximum velocity upper bound)} \\
g_2^x(x,\kappa) &= ((v_x^2 + v_y^2) \cdot \omega^2 - \mu g && \text{(Maximum centripetal acceleration upper bound)} \\
g_3^x(x,\kappa) &= -((v_x^2 + v_y^2) \cdot \omega^2) - \mu g && \text{(Maximum centripetal acceleration lower bound)} \\
g_4^x(x,\kappa) &= \delta - \delta_{\max} && \text{(Maximum steering angle upper bound)} \\
g_5^x(x,\kappa) &= -\delta - \delta_{\max} && \text{(Maximum steering angle lower bound)} \\
g_6^x(x,\kappa) &= \xi - \xi_{\max} && \text{(Maximum } \xi \text{ upper bound)} \\
g_7^x(x,\kappa) &= -\xi - \xi_{\max} && \text{(Maximum } \xi \text{ lower bound)} \\
g_8^x(x,\kappa) &= n - w/2 && \text{(Racetrack upper bound)} \\
g_9^x(x,\kappa) &= -n - w/2 && \text{(Racetrack upper lower)} \\
g_{10}^x(x,\kappa) &= (-\dot{s} + 1) && \text{(Enforce a minimum forward velocity of 1 m/s)}
\end{aligned} \tag{6}$$

and the input constraints

$$\begin{aligned}
g_1^u(u,\kappa) &= u_t - 1 && \text{(Max throttle)} \\
g_2^u(u,\kappa) &= -u_t - 1 && \text{(Min throttle)} \\
g_3^u(u,\kappa) &= u_\delta - u_{\delta,\max} && \text{(Max steering rate)} \\
g_4^u(u,\kappa) &= -u_\delta - u_{\delta,\max} && \text{(Min steering rate)}
\end{aligned} \tag{7}$$

Recall from Assignment 3 that we are interested in defining the dynamics of the system in terms of the intrinsic coordinates s rather than time, so that

$$\frac{dx}{ds}(s) = \frac{1}{\dot{s}(s)} f(x(s), u(s), \kappa(s))$$

In this way, the minimum lap time optimization problem becomes

$$\begin{aligned}
\text{minimize} \quad & \textstyle\int_0^S \frac{1}{\dot{s}(s)} ds \\
\text{subject to} \quad & \tfrac{dx}{ds}(s) = \tfrac{1}{\dot{s}(s)} f_t(x(s), u(s), \kappa(s)) = f_s(x(s), u(s), \kappa(s)) \\
& g_i^x(x(s), \kappa(s)) \leq 0 \quad \forall s \in [0, S] \quad \forall i = 1, \ldots, 11 \\
& g_i^u(u(s), \kappa(s)) \leq 0 \quad \forall s \in [0, S] \quad \forall i = 1, \ldots, 4 \\
& x(0) = x(S).
\end{aligned} \tag{8}$$

We can then write the continuous-time optimization problem above in discrete form, in a process that is typically called *transcription* in the literature. Namely, after discretizing the dynamics of the system with a given step ds (using e.g., Runge-Kutta or Euler integration), the problem in (8) becomes a nonlinear optimization problem of the form

$$\begin{aligned}
\text{minimize} \quad & \textstyle\sum_{k=0}^N \frac{1}{\dot{s}(s_k)} ds \\
\text{subject to} \quad & x_{k+1} = f_s^d(x_k, u_k, \kappa_k) \\
& g_i^x(x_k, \kappa_k) \leq 0 \quad \forall k \in [1, N] \quad \forall i = 1, \ldots, 10 \\
& g_i^u(u_k, \kappa_k) \leq 0 \quad \forall k \in [1, N] \quad \forall i = 1, \ldots, 4 \\
& x_0 = x_N.
\end{aligned}$$

which is now amenable to optimization. You will solve this problem in this assignment to find the best racing line you possibly can. The result of your optimization will then be a reference feedforward state and input trajectory

$$x^{\text{ff}} \in \mathbb{R}^{n \times N}, \quad u^{\text{ff}} \in \mathbb{R}^{n \times N}$$

which is not different from what you previously did in assignments 3 and 4, where you solved a much more difficult trajectory optimization problem.

Nonlinear Model Predictive Controller: When we have computed the fastest trajectory, we need our race car to track it using nonlinear model predictive control. Namely, your MPC

controller will look something like

$$\begin{array}{ll} \text{minimize} & \sum_{k=0}^{N-1} \left((x_k - x_k^{\text{ff}})^T Q (x_k - x_k^{\text{ff}}) + (u_k - u_k^{\text{ff}})^T R (u_k - u_k^{\text{ff}})\right) + (x_N - x_N^{\text{ff}})^T Q_T (x_N - x_N^{\text{ff}}) \\ \text{subject to} & x_{k+1} = f_s^d(x_k, u_k, \kappa_k) \\ & g_i^x(x_k, \kappa_k) \leq 0 \quad \forall k \in [1, N] \quad \forall i = 1, \ldots, 10 \\ & g_i^u(u_k, \kappa_k) \leq 0 \quad \forall k \in [1, N] \quad \forall i = 1, \ldots, 4. \end{array} \tag{9}$$

Note: When dealing with linear MPC, the MPC problem formulation involved a dynamics that was *linear* in the form

$$x_{k+1} = f_s^d(x_k, u_k, \kappa_k) = Ax_k + Bu_k + B_w \kappa_k.$$

Moreover, the constraints on the system state and input were linear in the form

$$g_i^x(x_k) = C_i^x x_k + b^x \leq 0, \quad g_i^u(u_k) = C_i^u x_k + b^u \leq 0,$$

But the structure looks really the same as you had for the linear case. The difference is only in that more theoretical guarantees can be given in the nonlinear case compared to the linear case. In this assignment, you will be learning several tricks that are applied to make nonlinear MPC faster and applicable in practice.

Variable	Name	Units	Variable	Name	Units
m	mass	kg	I_z	yaw inertia	kg·m^2
l_f, l_r	front/rear axle length	m	L	wheelbase	m
h_{com}	center of mass height	m	d_w	wheel to centerline dist.	m
κ_r	roll moment distribution	-	$F_{\text{drive,max}}$	max drive force	N
$F_{\text{brake,max}}$	max brake force	N	$\delta_{\max}$	max steering angle	rad
$\dot{\delta}_{\max}$	max steering rate	rad/s	$v_{\max}$	max velocity	m/s
μ	friction coefficient	-	C_d	drag coefficient	-
A_{ref}	reference area	m^2	ρ	air density	kg/m^3

Table 1: Vehicle parameters and units.

Design Task

In the assignment `.zip` file of the assignment, you will find the following Python files

1. File `pyproject.toml`: just a configuration file that **you don't have to touch**.
2. Folder **`racing`**: This folder represents the main package that we will use to visualize the optimal raceline and the performance of the MPC controller that you will design. You will **only** need to touch specific parts of this package according to the following instructions.
3. Folder **`competition_pack`**: This folder contains the reference racelines you have computed for your vehicle (see later to Question 2) and the scripts you will run to test your MPC controller.
4. File **`racing/model/kynematic_bicyle.py`**: This file contains the skeleton of the kinematic bicycle model. You will have to fill in the dynamics of your vehicle here according to Question 1.
5. File **`racing/racetrack/raceline_optimization.py`**: In this file, you will find the skeleton of the code to implement the raceline optimization as explored in Question 2.
6. File **`racing/mpc/mpc.py`**: In this file, you will find the skeleton of the code to implement the MPC controller for your race car as explored in Question 3.
7. Folder **`competition_pack/racetrack_name/vehicle_name`**: In the main folder, you will find several files of the form

   ```
   aragon/lamborghini/aragon_u_ref.npy
   aragon/lamborghini/aragon_x_ref.npy
   ```

   where, for example, `aragon` is the name of the racetrack and `lamborghini` is the brand of your vehicle. We will all use the same vehicle for this competition. In there, we have pre-saved some optimal racelines for you for each different track. This is because we want you to be able to take part in the competition even if you did not manage to run the raceline optimization.
8. File **`competition_pack/compare_racelines.py`**: This file is the one that you will run to save your optimal racelines. This script will pass through four different racetracks and find the optimal raceline for your vehicle. You **will not** have to modify this file. You should just run it and see the output of your raceline optimization.
9. File **`competition_pack/free_practice`**: This file is the one that you will run to test your MPC controller performance on the given racetrack and with your given optimal raceline. You **will not** have to modify this file unless you want to visualize some additional information on the raceline.

To run your code, you will need to install the `rancing` package in experimental mode, which you can do by calling the command `pip install -e .` in the same folder where the file `pyproject.toml` is located. The dependencies for this project will be updated automatically from this call.

Your task will be to answer the following questions by appropriately **implementing** the code, where required, and by **motivating** your design choices. It is suggested that you use plots from your code to answer the questions!

Question 1 : Setting up the dynamics

(a) Enter the file `racing/model/kynematic_bicyle.py` and check the method `__init__` of the class `KinematicBicycle`. First, fill in the nonlinear dynamics in (4) by writing the dynamics of each variable in the section

```
# Vehicle dynamics derivatives
heading_dot =
vx_dot =
vy_dot =
omega_dot =
delta_dot =
s_dot =
n_dot =
xi_dot =
```

All the parameters that you need for the vehicle are available in the form `self.parameters.parameters_name`. You can check the list of all these parameters in the class `Parameters` just below the class `KinematicBicycle`. The state and control variables, expressed as CasADi symbolic variables, are available as

```
self.heading_  = ca.SX.sym("heading") # heading
self.vx_       = ca.SX.sym("vx")      # longitudial
self.vy_       = ca.SX.sym("vy")      # lateral velocity
self.omega_    = ca.SX.sym("omega")   # heading rate
self.delta_    = ca.SX.sym("delta")   # steering angle
self.n_        = ca.SX.sym("n")       # lateral distance
self.xi_       = ca.SX.sym("xi")      # heading error angle

########## control inputs #################
self.throttle_    = ca.SX.sym("Fx")         # acceleration throttle [-1,1]
self.delta_dot_   = ca.SX.sym("delta_dot")  # steering rate
```

(b) Within the same class `KinematicBicycle` fill the constraints of the vehicle in the methods `get_state_constraints_function` and `get_input_constraints_function`. Namely, define the 12 expressions g_i^x in (6) and the four expressions g_i^u in (7), using the available state variables. For the sake of example, constraint g_4^x is defined as

```
gx_4 = self.delta_ -  self.parameters.max_steering_angle
```

Note: Once you have filled the constraint lists, you should see that a kind of normalization is applied to each constraint. The reason why this is done is that we would like all the constraints to be ***scaled*** so that all constraints should have a value in the range $[-1, 1]$. The motivation for this is that nonlinear solvers work much better when variables and constraints are scaled. For the sake of example, angles like δ and ξ live in the range $[-\pi, \pi]$, but the velocities v_x and v_y can have much bigger ranges like $[0, 30]$. Thus, it is always advised to scale your variables and constraints to make the problem more uniform for the solver. Oftentimes, this scaling will make a huge difference for your solver, so **it is always advised to scale your problem!**

Question 2 : Raceline optimization

Enter the file `racing/racetrack/raceline_optimization.py` and check the method function `compute_optimal_raceline_dynamic` and `compute_min_curvature_raceline_convex`. Our approach to compute the optimal raceline will be to first compute a good guess of the optimal raceline using convex optimization. You did this already in Assignment 2, and we implemented a version of that code in `compute_min_curvature_raceline_convex`, which you don't have to modify for now. After a good initial guess is found, we can use this to help our solver to find the optimal raceline.

(a) In the function `compute_optimal_raceline_dynamic` fill up the problem transcription by completing the `TODO` sections. As you will see, this is very similar to what you did for Linear MPC, just on a much bigger horizon and using nonlinear functions.

(b) (Bonus) Can you get some intuition on how the initial state guess is obtained by the convex optimization solution? The main goal here is to give a solution that at least respects the state constraints g_i^x and input constraints g_i^u. Describe in your own words how you think this is done, checking the code and knowing the dynamics of your bicycle model.

Once you have concluded this question, you a ready to visualize your optimized raceline. Just run the script **`compare_racelines.py`** and be patient. Your optimal racelines will be saved in the appropriate folder. You should also see an output from the solver of the form

```
This is casadi::Sqpmethod.

iter      objective    inf_pr    inf_du     ||d||  lg(rg) ls    info
   0   7.421445e+01  2.86e-01  1.19e-01  0.00e+00       -  0  -
   1   7.074428e+01  3.06e-02  1.77e+00  7.44e-01       -  1  -
   2   7.059106e+01  1.96e-02  1.60e+01  1.25e-01       -  1  -
   3   7.057391e+01  1.34e-04  3.01e+00  7.78e-03       -  1  -
   4   7.056685e+01  1.50e-07  2.09e-01  2.76e-04       -  1  -
   5   7.049949e+01  5.05e-07  1.11e-01  1.29e-03       -  1  -
   6   6.854869e+01  4.44e-05  8.84e-02  3.47e-02       -  1  - SOC -
   7   6.708719e+01  2.65e-05  8.71e-02  2.74e-02       -  1  - SOC -
   8   6.624553e+01  6.04e-06  9.49e-02  1.69e-02       -  1  - SOC -
```

where your objective is the raceline time, `inf_pr` is a scalar number that tells you how close you are to satisfying the given constraints, and `inf_du` tells you how close you are to optimal. All you have to understand is that the smaller `inf_pr` and `inf_du` are, the better the quality of your solution. In this example, your solver brought your raceline from 74 s to 66 s in just 8 steps of optimization. Not bad!

(c) Once the script **`compare_racelines.py`** has concluded, can you tell the difference between the initial guessed raceline (via convex optimization) and the optimal raceline? Describe what you see in your own words.

Note: In the function `compute_optimal_raceline_dynamic`, you can see the variables of your optimization problem have been scaled in the lines

```
# Define optimization variables
x_scaled = opti.variable(n_states, n_points)   # state variables
u_scaled = opti.variable(n_inputs, n_points-1) # control inputs
u        = u_scaled * input_scaling            # unscaled control inputs
x        = x_scaled * state_scaling            # unscaled state variables
```

to make sure your solver works with variables that are more or less bound in the range $[-1, 1]$. This is often **very** important in nonlinear optimization to even get a solution. The approach we took here for trajectory optimization is only a simplified version of a much bigger and complex optimization problem solved for real racing competitions. But scaling is something very important for solving general nonlinear problems numerically.

Question 3 : MPC

Finally, it is time to implement your MPC controller! Enter the file `racing/mpc/mpc.py` and look into the class `NMPC` and implement the missing sections in the method `_setup`, which is quite similar to the setup method you had developed in Assignment 4. In the implementation of your controller, you are free to

1. Include/exclude the state constraints that you deem necessary/unnecessary for your controller,
2. Change the cost function of your controller (e.g. adding a smooth input cost, changing the terminal cost),
3. Add slack variables to your controller where you find it necessary,
4. Change the CasADi solver that you use or the settings of the solver (we gave some good solver options, but you are free to experiment with more)

What you **must include** instead are the **input constraints to your model**. The reason why we let you omit state constraints is due to the fact that your trajectories will likely pass very close to the border of the racetrack, so having hard constraints there will likely lead your solver to failure. In the end, also the best pilots sometimes have to go off-road (see Valentino in the Figure below :)). Although keep in mind that you will receive some penalty for violating the constraints, which you can find in the rules of the competition in the next section.

Once you have finished filling up the class, the tuning of your controller will be done via the state and input cost matrices (expressed by the diagonal elements), the MPC horizon N and the discretization ds of your system (which we encourage to keep maximum at $ds = 0.5$.) For each retrack you will create a `.yaml` with the name `racetrack_par_mpc.yaml`. For example, you will have the files `assen_par_mpc.yaml`, `aragon_par_mpc.yaml`, `montecarlo_par_mpc.yaml` and `oval_par_mpc.yaml`.

```yaml
team_name: pizzalovers
car_model: lamborghini
Q: [10.0, 10.0, 10.0, 10.0, 100.0, 100.0, 100.0]
R: [1., 1.]
N: 20
ds: 0.5
```

In the implementation of your controller you are free to add slack variables to constraint and we encourage to do so. Indeed, in practice, constraints in Nonlinear MPC should almost always be softened using slack variables with a very high cost for not satisfying them.
We also encourage to omit the racetrack boundaries constraints. This is due to the fact that your optimal raceline will often pass very close to the boundary of the racetrack and your solver will have hard times finding a solution with this constraints. If your controller is well-tuned then your system should not exceed the boundary of the racetrack excessively. Even the best pilots, sometimes, have to overcome this the racetrack constraint

The competition

Training: Once you have solved all the tasks above you are now ready for the competition. You can test your MPC controller implementation by running the script `free_practice.py` where you will see an animated lap of your vehicle. You can also run the controller without any visualization if you want faster iterations and you are free to start the vehicle from different section of the racetrack to test the behaviour over specific sections. Before the competition your folder **must** contain

1. A tuning for the MPC controller for each racetrack as per Question 3,
2. The reference raceline for each racetrack using the same folder structure and naming given to you initially. If you did not solve the raceline optimization, please leave the references that were given to you initially.

If any of these are missing, in your submission, **you will unfortunately not be able to take part in the competition**.

The race: During the race, we will create a 1 vs 1 set of pairs where each team will compete against another team. We will organise the matches randomly the day before the competition. As it is for real system, the parameters of your vehicle will randomly be perturbed (inertia, μ, C_d). Still, both pairs of vehicles competing in one race will have the same perturbations to preserve fairness. Each competitor will receive a score equal to

$$\text{score} = \underbrace{\int_0^S \frac{1}{\dot{s}(s)} ds}_{\text{time}} + \underbrace{\lambda \int_0^S \max\{0, \left(\sum_{i=1}^{10} g_i^x(x(s), \kappa(s))\right)\} \, ds}_{\text{constraint violation penalty}} + \frac{\beta}{S} \underbrace{\int_0^S \text{cpu}(s) \, ds}_{\text{average solver-time penalty}} + \gamma N_{\text{fail}}$$

where $\lambda = 0.2$, $\beta = 1000$, $\gamma = 0.5$, $\text{cpu}(s)$ is the computational time taken by your MPC controller to provide the optimal solution, and N_{fail} is the number of times your solver fails during the racing. The competitor with the lowest score (minimum time + min violation + min cpu time + min number of failures) wins the competition and passes to the next round. For each competition, the racetrack over which the competition will take place will be randomly selected before the competition.

May MPC be with you! **Good Luck!**

Some stuff you can play around with

As in many other engineering sciences, there are not strict rules or formulas to design an optimal raceline, but here are some insights that can help you to take your racing to the next level:

(a) in the raceline optimization, you are welcome to increase the resolution of your trajectory optimization. You can do so by going into the script `compare_racelines.py` and modifying the resolution ds. Note that a higher resolution will also imply longer solution time.

(b) In the script `raceline_optimmization.py` you should check the solver settings inside the function `compute_optimal_raceline_dynamic`. Namely, you have the settings.

```
p_opts = {
 "qpsol": "osqp",
 "max_iter": 150,
 "hessian_approximation": "exact",
 "convexify_strategy": "regularize",
 "convexify_margin": 1e-3,
 "init_feasible": False,
```

```
        "elastic_mode": True,
        "second_order_corrections": True,
        "c1":1e-2,
        "beta":0.9,
        "tol_du": 2.5e-2, # how close to optimal do you want to be
        "tol_pr": 1e-4,   # how close to feasible do you want to be
        'gamma_0':0.1,
        "print_iteration": True, # set to true for debugging
        # QP options
        "qpsol_options.error_on_fail": 0,
        "qpsol_options.print_time": 0,
        "qpsol_options.verbose": 0,
        "qpsol_options.warm_start_primal": False,
        "qpsol_options.osqp.verbose": False,
        "qpsol_options.osqp.eps_abs": 1e-5,
        "qpsol_options.osqp.eps_rel": 1e-5,
        "qpsol_options.osqp.polish": True,

    }
```

While the meaning of every setting is outside the scope of the course (but we know of a good chatbot that could help you get some intuition on them), we highlight that the parameters "tol_du" and "tol_pr" define when your optimization stops. As mentioned before, the parameter "tol_du" will determine how much you push in the optimization. But be careful that pushing too much down might lead to the instability of the solver, which you will witness by non-convergent iterates. But you are free to play with these parameters.

(c) In the MPC solver, you should start by including no constraints at all with respect to the state and then add more constraints one by one. This will help you debug your code and understand which constraints are hard to satisfy. Note that your reference will probably be very, very close to the constraints boundary, as it is the optimized reference. So it is good that you add some slack constraints in there.

(d) In setting the MPC parameters, be mindful that your main source of directional control authority is the state δ, so you should maybe avoid penalizing it too much. Also, recall that a good position in the race track in terms of n and ξ might be more worth than matching the exact velocity of the vehicle. But you should experiment with this yourself.

(e) The solver settings of your MPC controller can be changed at your will. In general, the number of iterations determines how optimal your solution will be, but at the expense of the computational time.

(f) You can play with the parameters ds and N of your solver, but remember that this will have a big impact on the computational time. The farther you look, the better, but you might have to pay a price for it. This is the main MPC trade-off in practice.